



University of Southern Brittany
Vannes, France

Project Report
on
BigEarthNet LandCover Classification Using
Deep Learning with Distributed GPU-
Accelerated and Spark Framework

Big Data Course
Copernicus Master's in Digital Earth
(GeoData Science Track)

Group Members:
Rabina Twayana
Emanuel Farias Goulart

Submitted To:
Prof. Frederic Raimbault
(IRISA – UBS)

1 Feb 2026

ABSTRACT

The high volume of data present in Earth Observation requires the ability to process massive datasets. The high processing follows a shift from single-machine workflows to elastic, distributed cloud environments and cloud-based storage methods. This report documents the development and evaluation of a scalable, cloud-native Big Data pipeline and a custom U-Net deep learning architecture for pixel-wise Land-Use and Land-Cover (LULC) classification. Utilizing the BigEarthNet v2.0 dataset, the study leverages Apache Spark, Petastorm and Tensorflow on Amazon EMR for distributed preprocessing for multi-GPU training. The research specifically investigates the relationship between data volume and computational scalability. Experiments conducted demonstrate that while small data fractions (1%) suffer from parallelization overhead, larger datasets (up to 15%) achieve near-linear speedup, reducing training time from 56 minutes on a single GPU to approximately 16 minutes across four GPUs. Despite challenges with legacy libraries like Petastorm, the results confirm that a well-orchestrated distributed pipeline significantly enhances the throughput and feasibility of large-scale environmental mapping.

Keywords: PySpark, BigEarthNet, Deep Learning, Scalability, Speedup curves, Reproducibility

Table of Content

<i>ABSTRACT</i>	<i>i</i>
1. INTRODUCTION	1
1.1 Background	1
1.2 Objectives	1
1.3 BigEarthNet Datasets	1
2. TOOLS	2
3. METHODOLOGY	3
3.1 Exploratory Data Analysis and Metadata Extension	4
3.2 Data Preprocessing using Petastorm	4
3.3 Model Training and Performance Evaluation	5
4. TECHNICAL CHOICES	6
5. EXPERIMENT SETUP	8
6. RESULTS	9
6.1 Exploratory Data Analysis and Metadata Extension	9
6.2 Petastorm Dataset Creation	9
6.3 Model Training	10
6.4 Model Evaluation	11
6.5 Model Inference	12
6.6 Scalability Efficiency Analysis	13
7. DISCUSSIONS	15
8. CHALLENGES AND SOLUTIONS	15
9. CONCLUSION	17
REFERENCES	17

1. INTRODUCTION

1.1 Background

Technology evolution has pushed a data-centric revolution. Within our identification as society and the social construction given by the day-by-day of life, any singularity of time is composed of bits of data. Driven by the scalability of cloud computing, a second wave of digital transformation, led by Artificial Intelligence (AI), has emerged. This shift relies on massive volumes of data to train the next generation of digital products, resulting in significant real-world impacts

Today, these vast datasets are processed not on single machines, but across elastic, distributed clusters capable of handling high-volume gradient computations. The field of Earth Observation (EO) has mirrored this trend, optimizing its data sources to provide more accurate and impactful societal insights.

Navigating this landscape requires a skilled workforce in AI, distributed systems, and Big Data management. This report documents the development of a scalable big data pipeline and the construction of a deep-learning neural network from scratch.

The following report focuses on constructing from scratch a deep-learning neural network to train models capable of characterizing, on a pixel-wise level, land-use and land-cover labels to map larger areas and continents.

1.2 Objectives

The main objective of this project is to experiment with the efficiency and scalability of the Deep Learning (DL) algorithm in a distributed computing environment for the pixel-wise land cover classification on the large BigEarthNet dataset.

The secondary objectives of the project are as follows:

- To preprocess the input raster datasets and create and store petastorm datasets in S3 bucket.
- To develop the efficient DL model training and evaluation using Tensorflow MirroredStrategy.
- To access the scalability through the speedup curve for both preprocessing (based on number of executors) and model training (based on the number of GPUs) for different fractions of the datasets.

1.3 BigEarthNet Datasets

BigEarthNet v2.0 is a large scale benchmark dataset consisting of 549,488 pairs of Sentinel-1 and Sentinel-2 image patches (Clasen et al., 2025). Each image patch is associated with a pixel-level reference map and multiple land-cover class labels (i.e., multi-labels), derived from the most recent CORINE Land Cover (CLC) database of the year 2018. BigEarthNet v2.0 dataset was constructed by the Remote Sensing Image Analysis (RSiM) Group and the Database Systems and Information Management (DIMA) Group at the Technische Universität Berlin (TU Berlin). The dataset is collected between June 2017 and May 2018 across ten countries: Austria, Belgium, Finland, Ireland, Kosovo, Lithuania, Luxembourg, Portugal, Serbia, and Switzerland. BigEarthNet comprehends of 44 classes of Land-Use Land-Cover based on CLC. Additionally, it also provides the 19 classes by a simplified grouping of many detailed CLC classes, which is

majority recommended for scene or patch level classification. In a nutshell, this dataset enables training deep neural networks at scale, while accurately reflecting real-world land-cover diversity.

2. TOOLS

The tools that are used in this project are briefly described below:

a. Apache Spark

Apache Spark is the most widely used engine for scalable computing. It is a unified engine for large-scale data processing, analytics, and machine learning algorithms on single-node machines or clusters (The Apache Software Foundation, n.d.). The framework is based on the MapReduce paradigm, and it has numerous APIs implemented in Python, Java, and Scala (Polak, 2023).

b. PySpark

PySpark is the Python API for Apache Spark that allows access to the Spark distributed environment using Python. This allows users to build scalable data pipelines and machine learning workflows using familiar Python syntax. Within a PySpark application, *SparkContext* has a parameter called *_gateway*, which manages the connection interface to pass the Py4J application to the Spark Server running on JVM (Polak, 2023).

c. AWS- EMR (Amazon Web Service-Elastic MapReduce)

Amazon EMR is a big data processing service that provides tools and workflows to manage massive data in the cloud. With Amazon EMR, users can get access to web-based big data platforms, such as an S3 bucket, to process the massive amount of data using the available open-source tools like Trino, Apache Spark, and Apache Hive (Amazon Web Services, n.d.).

d. Amazon S3 (Simple Storage Service)

S3 is an industry-leading scalable storage service based on object storage technology that provides high security, durability, and low latency. The basic storage units of Amazon S3 are objects, which are organized into buckets. Each object is identified by a unique, and metadata can be used to flexibly organize the data. Data can be accessed easily through the Amazon Console or the powerful S3 API via an internet connection (Cloudian, n.d.).

e. Petastorm

Petastorm is an open-source data access library developed at Uber ATG. This library enables single machine or distributed training and evaluation of deep learning models directly from datasets in Apache Parquet format. For the current report, Petastorm is used in the process of creating Parquet files in the bucket to accelerate the dataloading process on a TensorFlow model (Uber ATG, 2019).

f. Tensorflow

Tensorflow is an open-source machine-learning and artificial intelligence library. It is designed to streamline the process of building, training, and deploying deep learning models across a wide range of applications. On the current report, the library is used to create the model architecture, run the training and scaling the training process by applying distributed computing strategies. The mirrow strategy is the technical choice.

g. S3FS

S3FS is a Pythonic file interface for S3. Built on top of boto3, it allows users to execute typical file-system style operations (cp, mv, cd). It should also be documented that S3F3 was also as a FUSE filesystem application backed by amazon web services simple storage service.

h. Pyarrow

PyArrow is a Python library that provides a high-performance interface for working with the Apache Arrow columnar memory format, enabling efficient in-memory data processing and interoperability across systems and languages. It is widely used to process, read, and write Parquet format files.

i. Rasterio

Rasterio is a Python library used for reading, writing, and processing geospatial raster data, such as satellite imagery. It provides tools to access multi-band raster files (e.g., GeoTIFFs) and convert them into NumPy arrays for numerical processing. Rasterio preserves spatial metadata, such as georeferencing and coordinate systems, while allowing efficient subsetting, resampling, and transformation of large raster datasets.

3. METHODOLOGY

Figure 1 represents the distributed data preprocessing (Spark + Petastorm) and deep learning based model training (UNet + MirroredStrategy) using multi-GPU in a AWS cloud environment. In this project, data analysis and metadata extension of BigEarthNet dataset was conducted initially, followed by preprocessing, model training, performance evaluation, and scalability analysis.

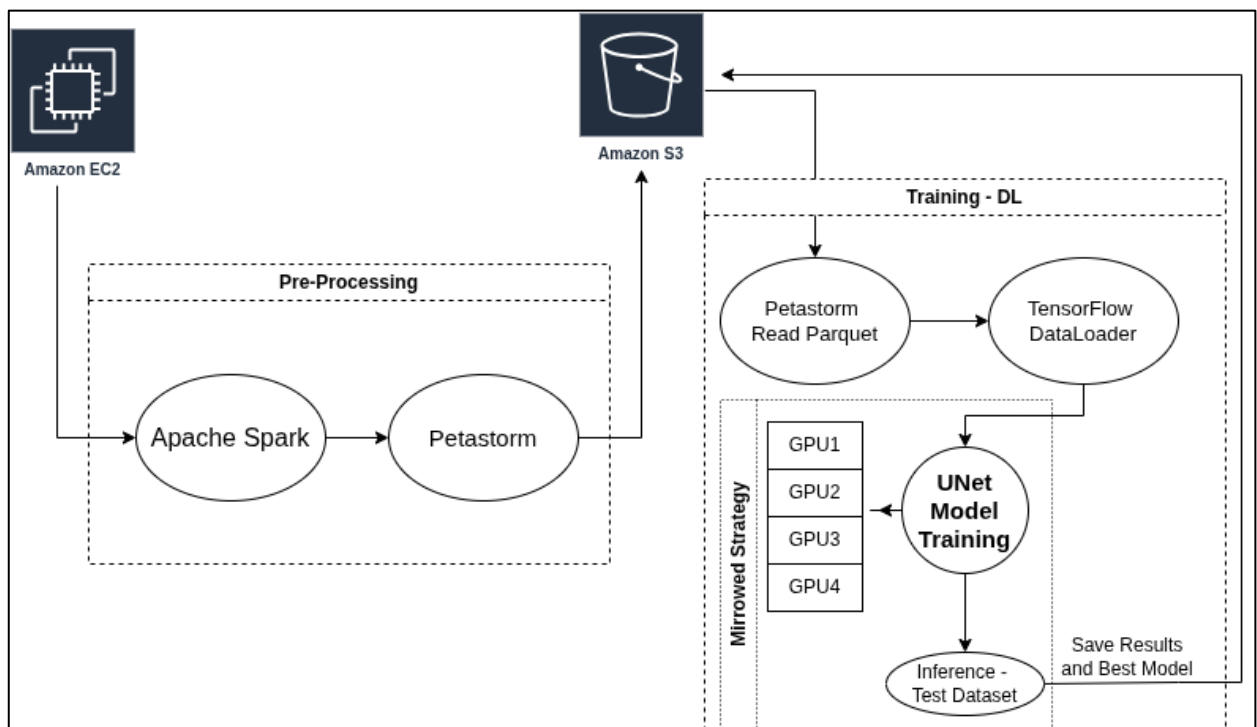


Figure 1: Preprocessing and training workflow for cloud-based distributed deep learning

The different stages of workflow is described below:

3.1 Exploratory Data Analysis and Metadata Extension

Exploratory data analysis was carried out to understand the structure and distribution of BigEarthNet dataset. The BigEarthNet dataset is accessed from the S3 bucket that includes Sentinel-1, Sentinel-2, reference dataset, and metadata. The metadata (.parquet file format) was first inspected to verify the schema, data types, and overall size of the dataset.

PyArrow and python was used for the initial metadata ingestion, and to create the structured link between the metadata and the physical assets stored on S3. To facilitate the data preprocessing and model training, the new metadata file was created and extended with additional columns to store the full absolute S3 bucket path of Sentinel-1 patch, Sentinel-2 patch, and reference map. Then, the enriched metadata is being written back to S3 as a Parquet file, enabling downstream deep learning pipelines, such as during **petastorm dataset creation**, to directly access the full S3 URIs.

A critical part of the this stage is ensuring "Data Integrity" before preprocessing begins. Since cloud storage can occasionally have missing objects, a distributed verification check is performed for random 1% of data using boto3 to check the existence of files.

3.2 Data Preprocessing using Petastorm

In this step, raw BigEarthNet dataset was processed, generate petastorm-compatible dataset and stored in S3, which is later used in deep learning step.

The pipeline began by initializing a SparkSession optimized for Amazon S3 data lakes using the S3A connector with the default AWS client handling to securely access to cloud storage. The extended metadata was accessed from S3 and filtered based on the "split" column to get the train, validation, and test sets specific to the fraction of data. Each sample is processed using Rasterio to read the Sentinel-1, Sentinel-2, and reference imagery. Then, Sentinel-1 and Sentinel-2 data were stacked together into a single multi-channel tensor, returning both the input images and corresponding labeled tensors.

The Petastorm dataset was created using the `materialize_dataset` function, which organizes the data according to a defined Unischema. Then, the created dataset was saved to S3 in Petastorm format, so that it can be directly accessed in the model training phase without additional preprocessing.

On the matter of scalability, the executor memory, driver memory, and the number of executors and cores are parameterized to ensure that preprocessing tasks are distributed across the cluster rather than processed sequentially, adding a level of parallelism necessary for increasing the fraction of data.

To ensure the preprocessing is efficient, two monitoring layers were set up: Spark Measure, to capture low-level task metrics, and a custom project logger, to wrap major blocks and provide a high-level view of which preprocessing stage consumes the most time. Finally, Logs are saved to S3.

3.3 Model Training and Performance Evaluation

The BigEarthNet land cover classification model was trained using a distributed deep learning workflow using Tensorflow and mirrored strategy across multiple GPUs.

a. Dataset & Data Loader

The train/val/test dataset is stored in a parquet format in S3 buckets in parquet format was read using the petastorm distributed dataset reader ‘make_reader’. Then, this dataset was converted into TensorFlow datasets with automatic sharding across GPUs. This also ensures that each GPU receives a unique portion of the data and does not see the same samples as other GPUs. Global batch size was computed based on the number of GPUs.

b. Model Architecture

To create the inference process and properly delineate on a pixel-level the segmentation task, the U-Net style convolutional neural network was implemented. Figure 2 exemplifies the Unet architecture proposed by Ronneberger et al. (2015). The network is based on a fully convolutional neural network whose architecture was modified and extended to work with fewer training images and to yield more precise segmentat

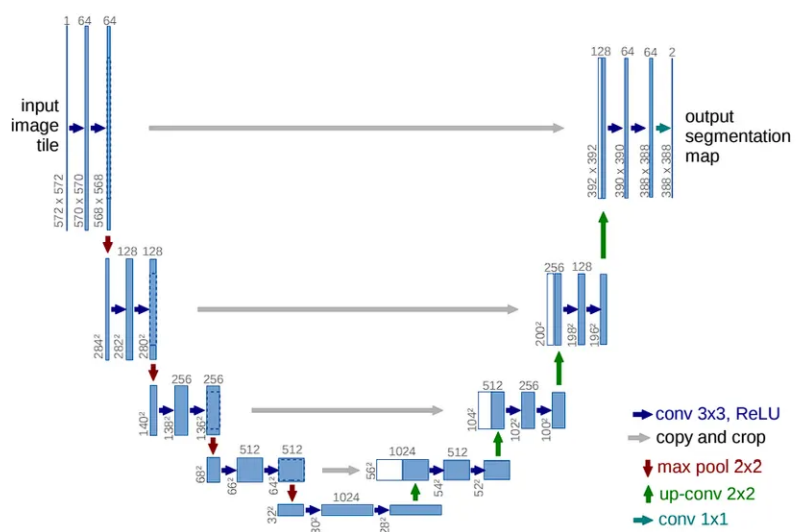


Figure 2: Unet Model Architecture: (Ronneberger et al., 2015)

two symmetric paths: a down-sampling path that captures image context through repeated convolution and pooling operations, and an up-sampling path that gradually reconstructs spatial resolution. Crucially, skip connections link corresponding layers in the encoder and decoder, allowing high-resolution features from early layers to be combined with up-sampled features, thereby improving segmentation accuracy and preserving fine details Ronneberger et al. (2015).

c. Model Training and Evaluation

The deep learning model was trained using the using a distributed multi-GPU setup with TensorFlow’s MirroredStrategy. The strategy copies the model into all GPUs available at the node, whereas each GPU holds a replica of the model’s weights. With the data loader process, each GPU receives a share, which means a different slide of the global batch. After the forward pass, each GPU computes the gradient, and before updating the weights, the Mirror Strategy uses an “All-Reduced” strategy to aggregate gradients across all GPUs. The synchronous update of every replica is done by the average gradient, ensuring the same weights at each new epoch.

Model performance was evaluated using sparse categorical accuracy on separate validation and test sets, and all training metrics, best model checkpoints, and logs were saved to S3 for reproducibility and analysis.

4. TECHNICAL CHOICES

This section explains the major technical choices made in implementing the distributed deep learning workflow:

a. API and Distributed Training Strategy: The code utilizes the Keras high-level API integrated with TensorFlow 2.x for building and training a deep learning model. To enable distributed training across multiple GPUs, TensorFlow Distribution Strategy called “tf.distribute.MirroredStrategy()”, was implemented, which handles synchronous data parallelism. The model and optimizer are instantiated within the `strategy.scope()`, ensuring that model variables are mirrored across all GPUs where gradients are calculated locally on each device at each step and then synchronized via an All-Reduce algorithm before the weights are updated simultaneously across all replicas. The code being identified within the strategy scope is an extremely important part in the process.

b. Data ingestion and sampling strategy: The pipeline is built for cloud-native, based mostly for dataloader on the Uber library, Petastorm, to bridge Parquet files on S3 buckets. The ingestion is done with pre-build functions of petastorm using “make_reader” to stream data directly from S3. Avoiding the need to download the dataset to the local disk. However, an important note here is that Petastorm is an outdated library, has not been updated in the last 2 years and therefore is generating many conflicts of library versions. All the conflicts were fixed; however, Petastorm does not present usability and large community engagement, no longer being anymore, the state-of-the-art of data ingestion.

The random sampling strategy without replacement was implemented to extract a varying fraction of the dataset for the experiment. Since the dataset was already spitted in train, test, and validation, corresponding fractions were sampled from each split without manipulation original ratios. For example, extracting 1% of the data involved selecting 1% of the training set, 1% of the validation set, and 1% of the test set, preserving the original split ratios while reducing the overall dataset size.

c. Feature Engineering: The input signature is (120, 120, 6), which is 120*120 pixels with 6 channels. Channels include Red, Green, Blue, NIR from Sentinel-2, and VV and VH from Sentinel-1. The fewer band selection was done to reduce the input dimension and memory footprint while capturing the most informative spectral features for the land cover classification task.

Reference maps have pixel values based on CLC (Corine Land Cover) spanning from 111 to 999 randomly. These values were mapped using LUT (Look-up-table) into 45 (0-44) unique values. This collapsing of IDs into a dense integer range is necessary to ensure the labels are compatible with the model's output layer and chosen loss function. Input images are cast to float32 during the generation phase to prepare for GPU-accelerated tensor operations.

d. Model choice and evaluation:

Since the Unet was coded from scratch and not trained weights were imported, a modification of the input size to 120x120 was applied instead of using the 540x540 standard size of Unet for trainable parameters.

UNet represents the architecture of choice, due to its large use and applications regarding segmentation tasks. U-Net's encoder-decoder structure with skip connections is ideal for preserving spatial context, necessary to improve task accuracy. A slight modification compared to the U-Net architecture published by Ronneberger et al. (2015) was at the bottleneck; our bottleneck has a dimension of 256 instead of 1024 as proposed originally. This was done to reduce the depth of the network, thereby reducing the complexity of the model for the experiment.

The **loss function** choice is sparse categorical cross-entropy, which is a loss function used in multi-class classification problems where the true labels are represented as integers (class indices) rather than one-hot encoded vectors. It is an equivalent to categorical cross-entropy but optimized for efficiency when dealing with a large number of classes. The loss function compares the predicted class probabilities with the true class indices to compute loss. The training was run for **10 epochs**, considering the multiple experiments that we had to make. A learning rate of **0.001** was selected for the Adam optimizer to support stable and efficient convergence, as it is a good starting point suggested for training deep learning models.

e. Memory, serialization, and shuffle tuning

To optimize memory usage, serialization and shuffle tuning, several practices were implemented in the training workflow. **Dataset prefetching** was applied using “`ds_tf.prefetch(tf.data.AUTOTUNE)`” to allow the CPU to prepare next batch while the GPU is computing the current batch, thereby improving the throughput. **Shuffle Tuning** was performed with “`ds_tf.shuffle(1000)`”. A buffer size of 1000 samples was used to balance randomness and memory consumption. Training history and model checkpoints are serialized to .csv and .keras formats locally and subsequently pushed to S3 for persistent storage, scalability and ease of automation through bash scripts.

f. Model Checkpointing and straggler mitigation:

The model is saved locally and then uploaded to the S3 after training completes. This ensures that the trained weights from the final epoch are preserved. For fault tolerance, the report checks that every epoch runs the ModelCheckpoint callback. If needed, training can be resumed later from the last saved .keras file to continue additional epochs.

Since Mirrored Strategy is synchronous, it is naturally sensitive to "stragglers" (one slow GPU slowing down the whole group). To mitigate this, the code standardized batch sizes per replica and uses an identical number of preprocessing workers (`workers_count=8`) for each GPU, ensuring the workload is perfectly balanced across all devices.

g. Execution Setup

The code was designed as a standalone Python script and run via a bash script, which allows setting up the parameters, such as the number of GPUs, the fraction of samples, S3 paths, and other settings necessary to run preprocessing and training experiments.

5. EXPERIMENT SETUP

For the experiments, EMR cluster initialized with one node with different numbers of GPUs was used to evaluate the scalability. The cluster is configured with EMR-managed scaling, allowing it to dynamically scale up to a maximum of eight instances depending on workload demand. During the pre-processing part, scalability was assessed on varying number of spark executors for different percentages of the total dataset. Similarly, for the model training, scalability is assessed on varying number of GPUs for varying fractions of dataset. The table 1 exemplifies the cluster configurations. The cluster consists of four GPUs, 192GB of RAM, eight virtual CPUs (totaling 48 vCores) and 900 GB SSD storage. This configuration provides sufficient computational and memory resources to access the pipelines throughput and execution time under different resource allocations.

Table 1: The configuration of the cluster environment

Clusters	Base Node	Max Instances	Max GPUs	RAM (GB)	CPU	vCore	SSD (GB)
EMR	1	8	4	192	8 vCPU	48	900

a. Experiment Configuration

Table 2 summarizes the resource configuration and hyperparameter settings used for data preprocessing and model training stages of the experiments. During the preprocessing stage, experiments were conducted using 1%, 5%, and 10% of the dataset. The number of Spark executors varied between 1, 5, and 8 to evaluate scalability. The preprocessing jobs were configured with 3 cores per executor, 16 GB of executor memory, and 8 GB of driver memory. This is because changing memory or cores per executor can introduce variability in Spark performance and complicate benchmarking workload scales across the executors.

Table 2: Experiment Configuration for Preprocessing and Model Training

Task	Percentage	Executors	Number of GPUs	Additional setting
Preprocessing	1%, 5%, 10%	1,4,8	-	Number of cores =3 Executor Memory= 16g Driver Memory= 8g
Model Training	1%, 3%, 5%, 10%, 15%	-	1, 2,3,4	LR= 0.001 (except 15% fraction with 3–4 GPUs: LR = $0.001 \times \text{num_gpus}$) GBS= $32 * \text{num_gpus}$
	3%	-	1,2,3,4	LR= $0.001 * \text{num_gpus}$ GBS= $32 * \text{num_gpus}$
	3%	-	1,2,3,4	LR= $0.001 * \sqrt{\text{num_gpus}}$ GBS= $32 * \text{num_gpus}$
LR=Learning Rate ; GBS= Global Batch Size; num_gpus= Number of GPU; sqrt= Square Root				

For the model training stage, different dataset sizes (1%, 3%, 5%, 10%, and 15%) were evaluated using 1 to 4 GPUs in order to evaluate the scalability. Although the experiment is focusing on the scalability side of deep learning model training in a distributed setting, the following three learning rate scaling strategies were tested as in the initial setting (Fixed learning rate), the performance (accuracy) of model decreases with increase in number of GPUs:

- a. Fixed learning rate, where the learning rate was set to 0.001 regardless of the number of GPUs.
- b. Linear scaling, where the learning rate was scaled proportionally to the number of GPUs ($LR = 0.001 \times \text{num_gpus}$).
- c. Square-root scaling, where the learning rate was scaled by the square root of the number of GPUs ($LR = 0.001 \times \sqrt{\text{num_gpus}}$).

In all training experiments, the global batch size (GBS) was scaled linearly with the number of GPUs ($GBS = 32 \times \text{num_gpus}$) to maintain a constant batch size per GPU.

6. RESULTS

6.1 Exploratory Data Analysis and Metadata Extension

The BigEarthNet metadata was loaded using pyarrow and the following information about the dataset was extracted.

- Total number of image patches: 480,038
- Columns: 8 (patch_id, labels, split, country, s1_name, s2v1_name, contains_seasonal_snow, contains_cloud_or_shadow)
- Train/validation/test count: 237871, 122342, 119825
- Train/validation/test ratio: 49.55 : 25.49 : 24.96
- Extended column name: s1_path, s2_path, ref_path
- Missing files count: 0 (Duration: 15 minutes for random 1 percent of data)

The new extended metadata was stored in S3 bucket for later steps and logs generated during this process were also saved in S3.

6.2 Petastorm Dataset Creation

Table 3 shows the pre-processing wall-clock execution time (in min) taken for converting BigEarthNet raw imagery into Petastorm datasets for different fractions of datasets (1%, 5%, 10%) over varying executors (1,4,8). The number of files is determined by dividing the total number of samples by 100, which results in a 28 MB Parquet file size each. The value 100 is selected based on the dataset size and the capacity of the cluster to process data efficiently in a distributed environment. Initially, when bigger parquet files were created, the processing time took longer. Thus, we decided to keep the file size small. The table illustrates how both data volume and parallelism affect processing time.

The execution time increases with an increase in data fractions as expected. For instance, when using a single executor, the preprocessing time rises from 9.77 minutes for the 1% dataset to further to 19.27 minutes for the 10% dataset. However, preprocessing does not benefit as expected with an increase in the number of executors. For the 10% fraction, the execution time only decreases from 19.27 minutes with one executor to 13.42 minutes with eight executors. The scaling is described in detail in the experimental finding sections below.

Table 3: Preprocessing Execution Time over different Executors

Fraction	Files Created (~28MB)			Execution Time (Seconds)		
	Train	Validation	Test	1 Executor	4 Executor	8 Executor
1%	24	12	12	9.77	7.27	4.35
5%	118	59	59	13.20	11.73	10.92
10%	236	119	119	19.27	15.92	13.42

6.3 Model Training

Table 4 summarizes the wall-clock execution times for different stages of the pipeline, which includes model training, model evaluation and the entire workflow. For model training, the execution time increases with an increase in dataset fraction. For example, approximately 4 minutes for 1% to 56 minutes for 15% data when using the single GPU. Data. In contrast, increasing the number of GPUs, the execution time reduces significantly. For instance, training the 15% dataset decreases from 56.18 minutes on 1 GPU to 15.83 minutes on 4 GPUs, demonstrating effective parallelization and scalability of the training process.

Evaluation times are measured in seconds and are much shorter than training times. The evaluation time increases slightly with an increase in dataset size and decreases marginally while increasing the number of GPUs. The total time, which includes reading data from S3, model training, model evaluation, and saving metrics and the best model, follows a similar trend to training time. This indicates that most model training time is the most dominant one. Saving models and metrics in S3 was taking less than 1 second regardless of fractions and the number of GPUs.

Table 4: Model Training Execution Time over GPUs

Category	Fraction	Number of GPUs			
		1	2	3	4
Model Training (Minutes) wall-clock time	1%	3.97	2.27	1.73	1.52
	3%	11.20	6.03	4.30	3.42
	5%	18.55	9.87	6.87	5.45
	10%	37.47	19.43	13.42	10.77
	15%	56.18	29.22	20.08	15.83
Model Evaluation (Seconds) wall-clock time	1%	6	4	4	1
	3%	13	9	8	8
	5%	20	12	11	12
	10%	39	22	21	21
	15%	59	32	30	31
Total Time including reading data, training model, evaluation, save metics and best model wall-clock time (Minutes)	1%	4.22	2.48	1.95	1.75
	3%	11.55	6.33	4.58	3.70
	5%	19.05	10.23	7.23	5.80

	10%	38.27	19.95	13.92	11.28
	15%	56.98	29.90	20.75	16.50

Figure 3 below exemplifies the training and validation accuracy and loss over training epochs (10) for the 15% data using 1 GPU. By analyzing the curves, it is possible to observe and analyze the convergence of the model. In addition, it can be observed that the model is not overfitting since the training accuracy is not spiking up when compared to validation, providing proof of a reliable and able generalization, with reduced bias.

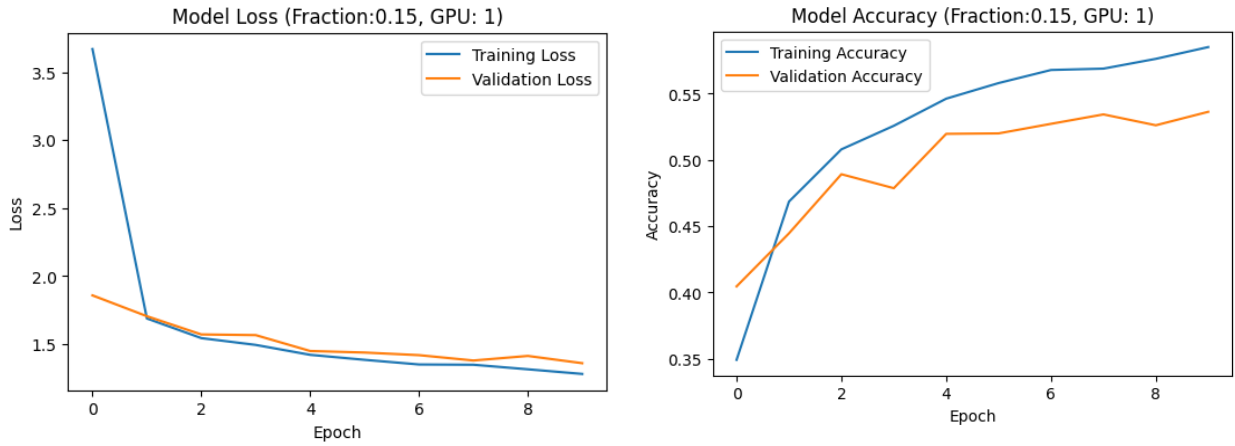


Figure 3: Model Training Loss and Accuracy over epochs for 15% data

6.4 Model Evaluation

Table 5 and Table 6 represent the training, validation, and test loss and accuracy, respectively, for different dataset fractions (1% to 15%) and GPU configurations (1–4 GPUs). It shows that increasing the dataset fraction generally reduces the loss across all splits, reflecting improved model accuracy with more data. For example, with 1 GPU, the training loss decreases from 2.240 to 1% of the dataset to 1.282 at 15%. In contrast, training accuracy increases from 30.22% in 1 percent of data to 58.52% in 15 percent of data. Slight increases in loss can be seen while increasing the number of GPUs. In all experiments, training loss is slightly lower than validation and test losses, reflecting standard behavior where the model fits the training data slightly better than unseen data.

Table 5: Train/Validation/Test Loss

GPUs	1			2			3			4		
Fraction	Train	Val	Test	Train	Val	Test	Train	Val	Test	Train	Val	Test
0.01	2.240	2.321	2.348	2.123	2.215	2.204	1.905	1.900	1.931	2.294	2.298	2.239
0.03	1.751	1.831	1.843	1.905	1.900	1.931	2.099	2.165	2.217	2.125	2.187	2.171
0.05	1.598	1.605	1.660	1.885	1.917	1.972	2.074	2.152	2.163	2.119	2.169	2.227
0.1	1.398	1.437	1.481	1.584	1.603	1.653	1.881	1.922	1.963	2.143	2.199	2.240
0.15	1.282	1.361	1.405	1.404	1.468	1.533	2.155	2.180	2.238	2.610	2.570	2.670

Table 6: Train/Validation/Test Accuracy

GPUs	1			2			3			4		
Fraction	Train	Val	Test	Train	Val	Test	Train	Val	Test	Train	Val	Test
0.01	30.2	28.33	30.52	32.27	29.76	31.62	31.45	30.13	32.1	30.12	26	30.49
0.03	46.9	42.89	43.41	41.51	39	39.31	33.31	28.85	29.46	30.88	21.69	24.86
0.05	49.6	47.68	47.18	41.45	37.86	38.09	33.14	30.72	31.35	33.25	31.38	29.47
0.1	55.12	51.69	51.90	49.45	46.86	46.45	41.76	37.71	38.39	33.18	28.99	29.30
0.15	58.52	53.63	53.83	54.94	51.45	50.92	32.34	28.14	28.58	14.74	12.97	11.99

Table 7 shows how model performance changes as the number of GPUs increases under three different learning-rate (LR) strategies: keeping the LR fixed at 0.001, scaling it linearly with the number of GPUs, and scaling it with the square root of the number of GPUs. The reported values (Train, Validation, and Test) represent model accuracy, where higher values indicate better performance. For 1 GPU, linear and square-root learning rate scaling are equivalent to the fixed learning rate; hence, experiments were not repeated. Both linear and square-root scaling of learning rate with respect to the number of GPU, outputs lower performance as compared to a fixed learning rate. Under the square-root learning rate scaling strategy, model performance remains relatively consistent as the number of GPUs increases. The Train, Validation, and Test metrics show only minor fluctuations from 2 to 4 GPUs, indicating that this scaling approach is more conservative and less sensitive to increased parallelism.

In contrast to square-root learning rate scaling, linear scaling results in lower performance with increase in GPU counts. Linear scaling introduce instability or over-aggressive updates, leading to degraded validation and test performance compared to the more conservative square-root scaling. This suggests that square-root scaling provides better generalization and stability under the given training setup. Nevertheless, the limited training duration of 10 epochs may have prevented full convergence of the Adam optimizer, influencing the comparative performance of the learning-rate scaling strategies.

Table 7: Accuracy (%) Comparison with Learning Rate Tuning (3% Dataset)

GPUs	Fixed LR = 0.001			LR = 0.001 × GPUs			LR = 0.001 × $\sqrt{\text{GPUs}}$		
	Train	Val	Test	Train	Val	Test	Train	Val	Test
1	49.10	47.27	46.88	-	-	-	-	-	-
2	41.51	39	39.31	33.17	29.33	31.04	33.72	28.74	30.77
3	33.31	28.85	29.46	30.72	21.36	23.34	30.24	22.42	25.56
4	30.88	21.69	24.86	14.21	16.96	15.32	31.91	28.92	28.09

6.5 Model Inference

The model checkpoints obtained in training process is saved in S3, which can be loaded and apply in new dataset for the prediction.

6.6 Scalability Efficiency Analysis

Speedup factor (equation as mentioned below) measures how much faster a task runs on multiple processors compared to a single processor.

$$Speedup = \frac{T_1}{T_n}$$

where:

T_1 = execution time of the task on 1 processor (single-threaded).

T_n = execution time of the task on n processors (multi-threaded or distributed)

a. Petastorm Dataset Creation Scaling

Figure 4 demonstrates the speedup curve of the preprocessing steps for different fractions of data over different numbers of executors. A significant gap between actual performance and the Ideal Speedup line can be seen, particularly as the data fraction increases. 1% fraction shows moderate scalability; the 5% and 10% fractions exhibit very low scaling, indicating that adding more executors does not scale as expected. For instance, the 1% sample achieves a ~2.24x improvement with 8 executors, but the 5% and 10% cases show only modest gains (~1.21-fold and 1.44-fold, respectively).

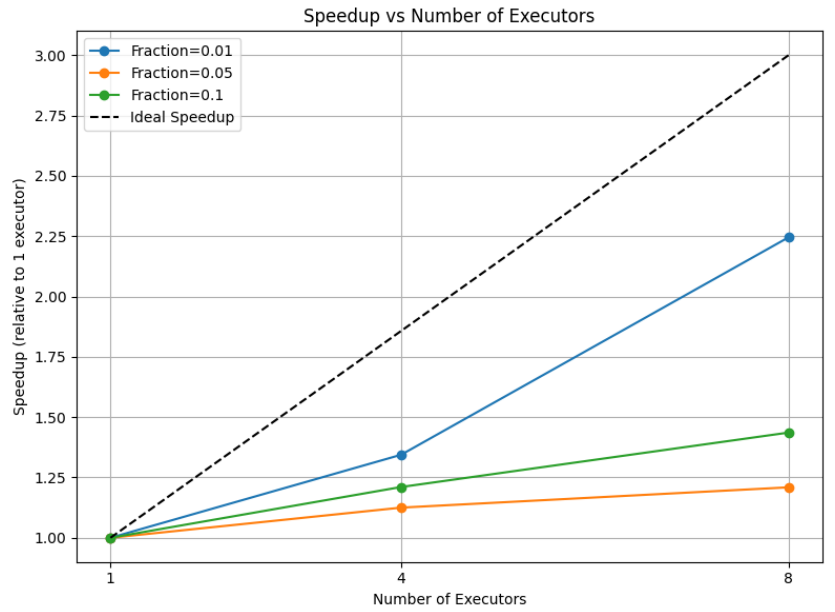


Figure 4: Preprocessing Speedup Curve

Table 8 presents the task metrics obtained for the 5% dataset when running preprocessing with 1, 4, and 8 executors. The executor CPU time remains extremely low (~1.5 minutes) irrespective of the number of executors, as compared to the executor run time (~4 hours). This indicates the CPU is not doing anything 99% of the time. This suggests that the tasks were predominantly I/O bound rather than compute-bound. In preprocessing, for each patch 7 (VV, VH, R, G, B, NIR, Ref) raster (.tiff) files were accessed from S3. This might cause massive network latency. 1% dataset demonstrates relatively better scaling. This could be because the total I/O volume was low in this case. As the dataset fraction increases (5% and 10%), the cumulative I/O cost dominates execution time, leading to diminishing returns from additional executors.

Table 8: Task Metrics for 5% Dataset

No of Executor	Task Duration	Executor Run Time	Executor CPU Time
1	3.90 h	3.89 h	1.5 min
4	3.87 h	3.86 h	1.5 min
8	3.83 h	3.83 h	1.4 min

b. Model Training Scaling

Figure 1 represents how the training speedup scales with the number of GPUs for different fractions of the dataset, normalized relative to a single GPU. The dashed line represents ideal linear speedup, where performance would increase proportionally with the number of GPUs.



Figure 5: Speed-up curve relative to GPUs for different dataset sizes

Table 9 complements this by providing the numerical speedup factors. For example, with 1% data, 2 GPUs achieve a 1.7x speedup and 4 GPUs 2.41x. When the dataset size is increased from 1% to 15% (15 $\times$ larger) and training is done on 4 GPUs, the execution time only increases by a factor of 3.45, rather than 15.

Table 9: Speedup Factor Table Relative to GPU-1

Fraction (%)	GPU-1	GPU-2	GPU-3	GPU-4
1	1.00	1.70	2.16	2.41
3	1.00	1.82	2.52	3.12
5	1.00	1.86	2.63	3.28
10	1.00	1.92	2.75	3.39
15	1.00	1.91	2.75	3.45

Figure and table shows that speedup increases with both GPU count and dataset size. An interesting observation is that no significant increase was observed for a fraction of 1% as compared to other fractions of data. In this case, since the dataset was still too small, likely the communication and synchronization overhead between GPUs exceeded the marginal benefits of

additional parallelism. In contrast, as the data volume increases, the speed up improves and moves closer to ideal scaling. For larger data fractions, the relationship between the number of GPUs and training throughput follows a linear distribution. This confirms that larger datasets benefit more effectively from multi-GPU parallel processing, as the increased computational density allows the system to approach near-linear scaling by minimizing the relative impact of communication latency.

Experiments beyond 15% dataset fraction were not conducted because scalability trends had already stabilized, and larger subsets would require significantly longer training time. Nevertheless, it can be concluded that the model benefits from multi-GPU parallelism with an increase in training throughput (large dataset). Although the scaling is below an ideal linear line up to 15%, this trend suggests that training on the full dataset would further improve GPU utilization, likely achieving near-linear speedup.

7. DISCUSSIONS

The experimental results conclusively demonstrate that 4 GPUs represent the optimal configuration for workloads comparable to those tested in this study. This configuration consistently delivered the highest speedup factors across all tested dataset fractions, reaching a peak speedup of approximately 3.4x for the 0.15 fraction. The performance stagnation observed in the preprocessing, where increasing the number of executors from 4 to 8 for fractions 0.05 and 0.1 resulted in significantly lower speedup. This highlights the importance of empirically determining the optimal parallelism level rather than assuming that more resources invariably yield better performance. In addition to this, the code logic implementation and package/library selection also play an important role in the scaling. For instance size and number of partition plays an important role in parallel processing.

Based on the current slopes up to 15% data, 4 GPUs would likely be the most optimal configuration for training the deep learning model. For 100% data, the limiting factor is not only the GPU but the VRAM capacity to prevent exceeding the memory. The experimental data show that larger datasets benefit more from higher parallelism, and the 4-GPU configuration will deliver the lowest execution time while likely utilizing the cluster's full 192 GB RAM capacity. If the dataset exceeds the memory of 4 GPUs, GPUs might need to scale up to 8 to simply fit the full dataset. For the preprocessing, optimization alternatives need to be sought to efficiently perform the I/O operation, thereby ensuring that increasing the executor count results in actual computational speedup.

8. CHALLENGES AND SOLUTIONS

Throughout the development and execution of the pipeline, several technical challenges emerged, each requiring specific mitigation strategies to ensure stable and effective experimentation.

a. The Parquet file size problem

When generating a Petastorm dataset, relying solely on the `rowgroup_size_mb` parameter to control file size leads to unexpectedly large Parquet files, such as 350 MB instead of the intended 126 MB. This occurs because Petastorm dynamically determines row groups based on row sizes and serialization overhead, which can result in fewer, larger files. Large files introduce challenges including slower I/O during training, higher memory usage, and reduced parallelism since Spark

has fewer partitions to process concurrently. To address this, the number of partitions was explicitly controlled in the DataFrame or RDD before passing it to `materialize_dataset`. Repartitioning the data ensures that each partition produces a manageable file size, improves parallelism during dataset creation, and reduces memory and I/O bottlenecks. Combining controlled partitioning with an appropriate `rowgroup_size_mb` value results in predictable, efficiently sized files and a more performant data pipeline.

b. Petastorm Sharding

Initially, dataset sharding was done using petastorm in a multi-GPU training setup. The default petastorm sharding works for 1% of data, but did not work when trying for 3% of data, resulting in memory exhaustion. The reason behind this is petastorm tries to load and cache all shards simultaneously which can exceed available memory. A Python generator was created to wrap the Petastorm reader, yielding (image, label) pairs for each sample. This generator was then converted into a TensorFlow dataset using `tf.data.Dataset.from_generator`, which streams data on-the-fly, avoiding the need to load the entire dataset into memory. By explicitly setting TensorFlow's `auto_shard_policy` to `DATA` and applying batching and prefetching, the dataset was evenly sharded across multiple GPUs with consistent batch sizes. This approach reduces memory usage, enables larger datasets to be processed efficiently, and ensures balanced GPU utilization during multi-GPU training.

c. UNET architecture problem

Initially, the simple UNET model architecture was setup with a total number of model parameters of 207k. The model training did not speed up with an increase in number of GPUs as expected. The primary reason for this could be the imbalance between computation and communication. When using multi-GPU data parallelism, the overhead of distributing inputs, gathering gradients, and synchronizing updates across GPUs becomes significant compared to the actual work each GPU is doing. Additionally, GPUs are optimized for heavy matrix operations, so small models fail to fully utilize GPU cores, leading to inefficient resource usage. To address this, the model size or complexity should be increased to increase computation per GPU, use larger per-GPU batch sizes, or employ gradient accumulation to simulate larger batches. In cases where the model is very small, it may be more effective to train on a single GPU rather than attempting multi-GPU scaling.

d. Learning Rate Tuning

An initial learning rate of 0.001 was used for all experiments, regardless of the number of GPUs. A slight decrease in accuracy was observed as the number of GPUs increased. To address this, two additional scaling strategies were evaluated: scaling the learning rate linearly with the number of GPUs ($\text{GPUs} \times \text{LR}$) and scaling it by the square root of the number of GPUs ($\text{LR} \times \sqrt{\text{GPUs}}$). However, neither approach resulted in noticeable improvements. Since the Adam optimizer was used, the learning rate is adaptively adjusted during training. Moreover, the experiments were limited to only 10 epochs, which was insufficient for full loss convergence. Therefore, increasing the number of training epochs is necessary for better model performance.

9. CONCLUSION

This project successfully demonstrated the implementation of an end-to-end, cloud-native pipeline for large-scale Land-Use and Land-Cover (LULC) classification. By integrating Apache Spark and Petastorm for distributed preprocessing and TensorFlow's MirroredStrategy for multi-GPU training, the workflow addressed the primary objective of managing the high-volume BigEarthNet v2.0 dataset within an Amazon EMR environment.

The experimental results highlight a critical "scalability threshold." While the benefits of distributed computing were marginal for smaller data fractions due to communication overhead, the system exhibited near-linear speedup as the data volume increased. Specifically, the reduction in training time for the 15% dataset, from 56 minutes on a single GPU to 16 minutes on four GPUs, confirms that distributed architectures are indispensable for processing at scale Earth Observation data.

Technically, the project navigated significant challenges, most notably the integration of Petastorm. While functional for bridging S3 Parquet files to TensorFlow, its status as a legacy library presented versioning conflicts, suggesting that future iterations should migrate toward more modern formats.

REFERENCES

- Amazon Web Services, Inc. (n.d.). *Amazon EMR*. Retrieved November 16, 2025, from <https://aws.amazon.com/emr/>
- Apache Spark. (n.d.). *Spark MLlib documentation*. <https://spark.apache.org/docs/latest/ml-guide.html>
- Clasen, K. N., Hackel, L., Burgert, T., Sumbul, G., Demir, B., & Markl, V. (2025). *reBEN: Refined BigEarthNet Dataset for Remote Sensing Image Analysis*. In *IEEE International Geoscience and Remote Sensing Symposium (IGARSS)*.
- Cloudian. (n.d.). *S3 Storage: How it Works, Use Cases and Tutorial*. Cloudian Blog. Retrieved November 16, 2025, from <https://cloudian.com/blog/s3-storage-behind-the-scenes/>
- Polak, A. (2023). *Scaling machine learning with Spark: Distributed ML with MLlib, TensorFlow, and PyTorch*. O'Reilly.
- Ronneberger, O., Fischer, P., & Brox, T. (2015). *U-Net: Convolutional networks for biomedical image segmentation*. arXiv. <https://arxiv.org/abs/1505.04597>
- Uber ATG. (2019). *Petastorm: Efficient data access for deep learning from Parquet datasets* [Computer software]. GitHub. <https://github.com/uber/petastorm>

AI Disclaimer: Used for rephrasing and rewording some sentences to enhance clarity and conciseness.